



Numbers, Variables & Expressions

Python Mastery — Part 1: Foundations · Week 2

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap

Where we got to last week

Last week, in one breath

- ▶ You installed **Python 3.14**, **uv**, **Ruff**, **VS Code**
- ▶ You met the **REPL** — type a line, get an answer
- ▶ You ran a script **two ways**: `python hello.py` and `python -m hello`
- ▶ You scaffolded **taskapp** with `uv init` and tidied it with **Ruff**

Today we use that exact REPL and that exact project.

Today's goal

By the end of this session you'll be able to:

- ▶ Use Python's number types: `int`, `float`, `complex`, `bool`
- ▶ Tell apart `/`, `//`, and `%`, and use `**`
- ▶ Predict results from **operator precedence**
- ▶ Bind and rebind **variables**; use `None`
- ▶ Defend against **floating-point** surprises

Numbers & operators

The value layer of every program

The four numeric types

- ▶ `int` — whole numbers, **unlimited** size: `42`, `-7`, `10**100`
- ▶ `float` — decimals: `3.14`, `2.0`, `1.5e3`
- ▶ `complex` — real + imaginary: `2 + 3j` (rarely needed, good to know)
- ▶ `bool` — `True` / `False`, and `bool` is a kind of `int`

```
>>> True + True
2
>>> type(42), type(3.14)
(<class 'int'>, <class 'float'>)
```

Arithmetic operators

Operator	Means	7 ? 2
+ - *	add, subtract, multiply	—
/	true division → always float	3.5
//	floor division → rounds down	3
%	modulo → remainder	1
**	power	49

/ always gives a float , even $4 / 2 \rightarrow 2.0$.

// and % travel together

```
>>> 17 // 5
3
>>> 17 % 5
2
>>> 3 * 5 + 2
17
```

- ▶ // = how many whole times it fits
- ▶ % = what's left over
- ▶ Together they split a number cleanly — useful for time, change, grouping

Mixing types & precedence

- ▶ Mix `int` and `float` → result **widens** to `float`: `2 + 3.0` → `5.0`
- ▶ Precedence (highest first): `**` → `*` `/` `//` `%` → `+` `-`
- ▶ Same level runs **left to right**
- ▶ **When in doubt, use parentheses** — they cost nothing and read clearly

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

Let's use Python as a calculator.

Watch this — Demo 3

REPL calculator & precedence

- ▶ `7 / 2`, `7 // 2`, `7 % 2`, `2 ** 10`
- ▶ `(2 + 3) * 4` vs `2 + 3 * 4`
- ▶ Mix `int` and `float` → watch the result **type**
- ▶ `x = 5; x += 3` (augmented assignment)
- ▶ `a, b = 1, 2` (multiple assignment)

The operators, side by side

```
>>> 7 / 2
```

```
3.5
```

```
>>> 7 // 2
```

```
3
```

```
>>> 7 % 2
```

```
1
```

```
>>> 2 ** 10
```

```
1024
```

/ floats · // floors · % remainder · ** power

Precedence in action

```
>>> 2 + 3 * 4
```

```
14
```

```
>>> (2 + 3) * 4
```

```
20
```

```
>>> 2 ** 3 ** 2
```

```
512
```

- ▶ `*` binds tighter than `+`
- ▶ `**` runs **right to left**: `2 ** (3 ** 2)`

What you just saw

- ▶ `/` always returns a `float`; `//` and `%` return whole numbers for ints
- ▶ `**` is power, and it groups **right to left**
- ▶ Mixing `int` and `float` widens the answer to `float`
- ▶ Parentheses override precedence — and aid readability

Variables & **None**

Names that point at values

Variables: names bind to objects

- ▶ `=` does **not** "copy into a box" — it **binds a name to an object**
- ▶ A name can be **rebound** any time, even to a different type

```
>>> x = 5
>>> x = "now a string"
```

- ▶ **Augmented:** `x += 3` means `x = x + 3` (also `-=`, `*=`, `//=`, ...)
- ▶ **Multiple:** `a, b = 1, 2` · swap with `a, b = b, a`

None — the absence of a value

- ▶ A special object meaning "nothing here yet"
- ▶ Not `0`, not `""`, not `False` — its own thing
- ▶ Used for "no result", "not set", default placeholders

```
>>> result = None
>>> result is None
True
```

Test it with `is None`, not `== None`.

Input is always text

- ▶ `input(...)` **always** returns a `str` — even `"42"`
- ▶ Convert before you compute: `int(...)` or `float(...)`

```
>>> age = input("Age? ")      # you type 42
>>> age + 1
TypeError: can only concatenate str ...
>>> int(age) + 1
43
```

Forgetting to cast is the #1 beginner bug here.

Now — the famous floating-point surprise.

Watch this — Demo 4

Floating-point surprises & fixes

- ▶ `0.1 + 0.2` → **not** `0.3`
- ▶ Why: decimals stored as **binary fractions**
- ▶ Fix 1: `round(0.1 + 0.2, 2)`
- ▶ Fix 2: `math.isclose(0.1 + 0.2, 0.3)`
- ▶ Then: read two numbers, cast, compute a **tip**

The surprise

```
>>> 0.1 + 0.2  
0.30000000000000004  
>>> 0.1 + 0.2 == 0.3  
False
```

- ▶ Floats are **approximate** — most decimals can't be stored exactly in binary
- ▶ This is true in nearly every language, not just Python

The three defenses

```
>>> round(0.1 + 0.2, 2)
0.3
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>> f"{0.1 + 0.2:.2f}"
'0.30'
```

- ▶ **round** for a rounded value · **isclose** to compare · **format** to display

What you just saw

- ▶ `0.1 + 0.2` is `0.30000000000000004` — floats are approximate
- ▶ **Never** compare floats with `==`; use `math.isclose`
- ▶ **Round** for a clean value, **format** for clean display
- ▶ `input()` gives text; cast with `int()` / `float()` before math

Q&A + gotchas

The mistakes that bite beginners

Common arithmetic gotchas

- ▶ `5 / 2` is `2.5` , **not** `2` — `/` always floats
- ▶ `//` rounds **toward negative infinity**: `-7 // 2` is `-4` , not `-3`
- ▶ `int("3.5")` **errors** — cast text to `float` first, then `int`
- ▶ Dividing by zero raises **`ZeroDivisionError`** — guard against it
- ▶ Comparing floats with `==` ? Use `math.isclose` instead

A first touch of `math`

```
>>> import math
>>> math.sqrt(144)
12.0
>>> math.floor(3.9), math.ceil(3.1)
(3, 4)
>>> math.pi
3.141592653589793
```

The standard library already has the maths you need — just `import math`.

Wrap-up

This week's homework

Lab 2 — a safe little calculator

1. Write `calculator.py` beside your `taskapp`
2. Read **two numbers** with `input()`, cast with `float()`
3. Print **sum, difference, floor-division, remainder, percentage**
4. **Round** results sensibly — no float noise
5. **Submit:** the script + its output for the given sample inputs

Mirror **Demo 3** for the operators, **Demo 4** for the rounding.

Next week

- ▶ **Week 3 — Strings & Text**
- ▶ We turn numbers into clean, formatted output with **f-strings**
- ▶ You'll need today's casting and rounding to format real values

See you in the lab.

Questions?